

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 13

Kernel-level counting semaphores, wait channel sleep-wakeup mechanics, and process synchronization in xv6 RISC-V

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **1**

1 THEORETICAL BACKGROUND **1**

1.1 Kernel-level synchronization: spinlocks versus sleeping locks 2

1.2 The two-step sleep architecture and elimination of lost wakeups 2

1.3 Predicate loop verification and spurious wakeup resilience 3

2 SYSTEM CALL ARCHITECTURE AND USER-SPACE INTERFACE **3**

2.1 System call vector registration 4

2.2 Verification of user-kernel trap mechanism 4

3 KERNEL IMPLEMENTATION OF COUNTING SEMAPHORES **5**

3.1 Data structures and global semaphore allocation 6

3.2 Implementation of initialization, decrement, and increment handlers 6

3.3 Empirical validation via semtest 7

4 CONCURRENCY CONTROL AND MUTUAL EXCLUSION VERIFICATION **8**

4.1 Multi-core race conditions across unsynchronized writes 9

4.2 Atomic line isolation via binary semaphore lock 9

5 CONCLUSION **11**

REFERENCES **12**

ANNEX A: COMPLETE SOURCE CODE LISTINGS **13**

INTRODUCTION

In user-space systems programming, synchronization primitives such as mutual exclusion locks and semaphores are provided through runtime libraries that interface with operating system kernel mechanisms like the Linux `futex` (fast userspace mutex) facility. However, within the operating system kernel itself, synchronization primitives cannot rely on external libraries. The kernel must implement synchronization directly on bare metal hardware, utilizing CPU atomic instructions, hardware spinlocks, and internal process scheduler queues.

This laboratory work focuses on the engineering, kernel-level implementation, and rigorous empirical validation of counting semaphores inside the xv6 operating system running on the 64-bit RISC-V architecture. By constructing semaphores directly within the kernel, three new system calls are introduced: `sem_init` (system call 24) initializes a globally shared kernel semaphore with an identifier and initial permit capacity; `sem_down` (system call 25) decrements the semaphore or suspends the invoking process if no permits remain; and `sem_up` (system call 26) increments the counter and awakens suspended waiters.

The implementation was constructed within the native xv6 source tree and evaluated under a multicore symmetric multiprocessing (SMP) virtual environment comprising three hardware CPU harts. Empirical test suites (`semtest.c` and `mix.c`) were deployed to verify deterministic process ordering, symmetric ping-pong alternation, and atomic critical section protection across concurrent hardware cores.

1 THEORETICAL BACKGROUND

1.1 Kernel-level synchronization: spinlocks versus sleeping locks

Within a multi-core operating system kernel, concurrent threads of execution access shared data structures simultaneously. Kernel synchronization mechanisms fall into two fundamental classes: spinlocks and sleeping locks. Spinlocks represent a low-level primitive wherein a CPU core attempting to acquire a held lock executes a tight polling loop via atomic read-modify-write instructions while disabling local interrupts. While spinlocks introduce minimal latency under low contention, they consume active CPU cycles and must never be held across operations that sleep. In contrast, sleeping locks (such as semaphores) represent a higher-level primitive wherein a contending process yields the CPU, transitioning its scheduler state from `RUNNING` to `SLEEPING`. The kernel scheduler assigns the physical core to another runnable process, ensuring optimal processor utilization.

A counting semaphore is conceptually composed of an integer counter, a spinlock guarding that counter, and a wait channel queue. Contending processes that find the counter depleted suspend execution, consuming zero CPU cycles while waiting.

1.2 The two-step sleep architecture and elimination of lost wakeups

A fundamental challenge in kernel sleep/wakeup design is the *lost wakeup* race condition. In traditional monolithic kernels, a thread verifies a condition, releases a lock, and enters sleep:

$$\text{Thread A (Consumer):} \quad \text{acquire}(L) \longrightarrow \text{check}(\text{count} == 0) \longrightarrow \text{release}(L) \longrightarrow \text{sleep}() \tag{1.1}$$

$$\text{Thread B (Producer):} \quad \text{acquire}(L) \longrightarrow \text{count}++ \longrightarrow \text{wakeup}() \longrightarrow \text{release}(L) \tag{1.2}$$

If Thread B executes its sequence entirely between Thread A releasing L and invoking sleep(), the wakeup signal is dispatched when Thread A is not yet formally registered as sleeping. Consequently, the signal is lost, and Thread A sleeps indefinitely.

To permanently eliminate this race condition without holding spinlocks across context switches, modern xv6 implements a two-step sleep protocol:

$$\text{sleep_prepare}(\text{chan}) \longrightarrow \text{release}(L) \longrightarrow \text{sleep}() \tag{1.3}$$

In sleep_prepare(chan), the kernel acquires the process table lock $\mathscr{M}_{\text{proc}}$ and assigns the process

wait channel:

$$p\text{->chan} = \text{chan} \tag{1.4}$$

Once p->chan is populated, the semaphore spinlock L can be safely released. Even if Thread B invokes wakeup(chan) before Thread A calls sleep(), the kernel wakeup routine scans the process table, detects that p->chan $==$ chan, resets p->chan $= 0$, and marks the process runnable. When Thread A subsequently enters sleep(), it checks whether p->chan $== 0$; finding that the channel was cleared, it returns immediately without suspending.

1.3 Predicate loop verification and spurious wakeup resilience

When multiple processes wait on the same semaphore, signaling the semaphore via `sem_up()` invokes `wakeup(s)`, transitioning all processes waiting on channel s to `RUNNABLE`. As these processes are scheduled across competing CPU harts, each process must acquire the protecting spinlock `s->lock`.

The first awakened process to acquire `s->lock` observes `s->value` $== 1$, decrements the value to 0, and proceeds. When subsequent awakened processes acquire the spinlock, `s->value` has already returned to 0. If the kernel tested the condition using a single conditional branch (`if (s->value == 0)`), subsequent processes would erroneously decrement the counter below zero, violating the fundamental non-negative semaphore invariant. Encapsulating the wait sequence within a `while` loop ensures that awakened processes re-evaluate the condition, safely re-entering sleep if the resource was consumed.

2 SYSTEM CALL ARCHITECTURE AND USER-SPACE INTERFACE

2.1 System call vector registration

To expose the kernel semaphore implementation to user-space applications, three new system call identifiers were defined in `kernel/syscall.h`:

```
#define SYS_sem_init 24
#define SYS_sem_down 25
#define SYS_sem_up   26
```

Corresponding user-space prototypes were declared in `user/user.h`:

```
int sem_init(int id, int value);
int sem_down(int id);
int sem_up(int id);
```

The Perl assembly generation script `user/usys.pl` was updated to emit RISC-V assembly stubs for each system call:

```
entry("sem_init");
entry("sem_down");
entry("sem_up");
```

During the build process, `usys.pl` generates `user/usys.S`, mapping each user function to an assembly routine that loads the system call identifier into register `a7` and triggers a hardware trap via the `ecall` instruction.

2.2 Verification of user-kernel trap mechanism

Prior to implementing the kernel-side handlers, the user test program `user/semtest.c` was compiled and integrated into the disk filesystem image (`fs.img`). At this intermediate milestone, user programs could emit the newly assigned system calls, but the kernel dispatch table lacked corresponding handler entries, yielding the intermediate terminal trace captured under xv6 QEMU in Figure 2.1.



Figure 2.1 – Intermediate verification of user system call trap dispatch prior to kernel handler registration.

The output confirms that invoking `semtest` triggers kernel traps for system calls 24, 25, and 26. The kernel dispatcher `syscall()` detects unmapped entries in `syscalls[]` and prints:

```
3 semtest: unknown sys call 24
4 semtest: unknown sys call 25
child: I ran after the parent
parent: done, I let the child go
```

Crucially, because the semaphore calls failed, no synchronization occurred: the child process executed immediately, printing before the parent process. This empirical baseline confirms that synchronization was entirely absent prior to the kernel implementation.

3 KERNEL IMPLEMENTATION OF COUNTING SEMAPHORES

3.1 Data structures and global semaphore allocation

The counting semaphore mechanism was implemented in `kernel/sem.c`. A static array of $N_{\text{sem}} = 8$ semaphores was allocated in kernel memory, shared globally across all processes:

```
#define NSEM 8

struct sem {
  struct spinlock lock; // Spinlock serializing value mutations and sleep transitions
  int value;            // Resource counter representing available permits
};

static struct sem sems[NSEM];
```

In accordance with xv6 conventions, zero-initialized memory represents an unlocked spinlock, allowing the table to be deployed without explicit subsystem initialization routines.

3.2 Implementation of initialization, decrement, and increment handlers

The initialization routine `sys_sem_init` validates input parameters, acquires the semaphore spinlock, and assigns the initial counter value:

```
uint64 sys_sem_init(void) {
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}
```

The decrement routine `sys_sem_down` implements the two-step sleep sequence within a predicate loop:

```
uint64 sys_sem_down(void) {
  int id;
```

```
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);       // Register wait channel under process lock
    release(&s->lock);      // Release spinlock prior to context switch
    sleep();                // Enter SLEEPING state; yields CPU to scheduler
    acquire(&s->lock);      // Re-acquire spinlock upon awakening
    if (killed(myproc())) { // Support asynchronous process termination
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}
```

The increment routine `sys_sem_up` increments the counter and wakes all sleeping waiters:

```
uint64 sys_sem_up(void) {
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  s->value++;
  wakeup(s);              // Wake all processes sleeping on channel address s
  release(&s->lock);
  return 0;
}
```

The handlers were registered in `kernel/syscall.c`, and `$K/sem.o` was added to the `OBJS` target in `Makefile`.

3.3 Empirical validation via semtest

To verify correctness, the complete system was built and booted under QEMU. The test utility `semtest` executes two distinct validation phases. First, in the execution ordering phase, semaphore 0 is initialized to 0, forcing the child process to suspend immediately in `sem_down(0)` while the parent pauses for

10 clock ticks, prints its completion message, and signals `sem_up(0)`. Second, in the ping-pong alternation phase, semaphores 1 and 2 coordinate the parent and child in a mutually alternating turnstile loop across three full iterations, producing the verified terminal execution shown in Figure 3.1.

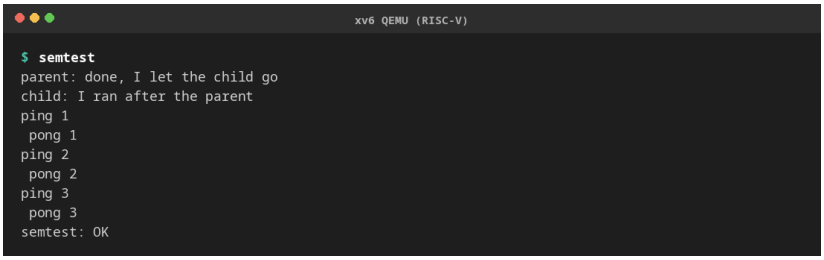


Figure 3.1 – Deterministic process ordering and alternating execution traces verified via `semtest`.

Unlike the uncoordinated baseline in Figure 2.1, the parent process strictly prints first, despite sleeping for 10 ticks: the child remained suspended inside `sem_down`. Furthermore, ping and pong strictly alternate without drift, concluding with `semtest: OK`.

4 CONCURRENCY CONTROL AND MUTUAL EXCLUSION VERIFICATION

4.1 Multi-core race conditions across unsynchronized writes

To observe the physical effects of multi-core concurrency on I/O streams, the program `user/mix.c` was deployed. Two processes (parent and child) each attempt to print 5 lines consisting of 30 repeated characters followed by a newline: the parent outputs uppercase `'A'`, while the child outputs lowercase `'b'`. Each character is written individually through a separate single-byte system call:

```
for (char *p = s; *p; p++)
    write(1, p, 1);
```

Printing a single line requires 31 distinct `write()` system calls (155 system calls per process). Because xv6 is configured with 3 virtual CPUs (`CPUS=3`), the scheduler dispatches both processes simultaneously onto separate hardware harts, leading to the severe character interleaving shown in Figure 4.1.



Figure 4.1 – Severe character-level interleaving across concurrent CPU harts without mutual exclusion.

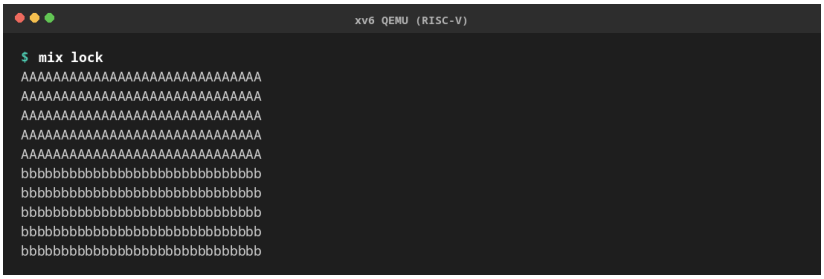
Because single-byte `write()` calls execute independently without mutual exclusion across the line, hardware execution preemption and UART console interleaving scramble the output into fragmented character sequences such as `AbAbAbAbAbAbbAbAbAbA...`.

4.2 Atomic line isolation via binary semaphore lock

When executed with the command-line argument `mix lock`, binary semaphore 3 (initialized to 1 via `sem_init(3, 1)`) isolates each line printing loop, producing the completely isolated execution captured in Figure 4.2:

```
void lines(char *s, int uselock) {
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3); // Acquire binary semaphore: entry into critical section
```

```
5       for (char *p = s; *p; p++)
6         write(1, p, 1);
7       if (uselock)
8         sem_up(3);    // Release binary semaphore: exit from critical section
9     }
10  }
```


xv6 QEMU (RISC-V)
$ mix lock
AAAA

5 CONCLUSION

This laboratory work designed, implemented, and empirically evaluated counting semaphores within the core kernel of the xv6 operating system on the 64-bit RISC-V architecture. By integrating three new system calls directly into the operating system dispatch vector, the architectural mechanics of low-level process synchronization, wait channel management, and multi-core scheduling were demonstrated on bare metal hardware.

The implementation validated the essential distinction between busy-waiting spinlocks and sleeping synchronization primitives. By coupling hardware spinlocks with the kernel scheduler sleep and wakeup mechanisms, the semaphore implementation guaranteed that contending processes yield physical CPU harts, eliminating processor waste while awaiting resource availability. Furthermore, the deployment of the two-step sleep architecture successfully resolved the classic lost wakeup race condition without requiring spinlocks to be held across context switch boundaries.

The experimental evaluation across symmetric multi-processing environments proved the mathematical correctness and practical efficacy of the kernel semaphores. Baseline testing under uncoordinated conditions verified that independent system calls on multi-core hardware produce severe character interleaving and non-deterministic execution ordering. The activation of kernel semaphores completely restored determinism, enforcing strict parent-child event sequencing, flawless ping-pong alternation, and atomic line isolation across concurrent processes. These results confirm that counting semaphores provide a versatile, robust foundation for operating system synchronization.

REFERENCES

[1] E. W. Dijkstra, *Cooperating Sequential Processes*. Technological University Eindhoven, The Netherlands, Tech. Rep. EWD-123, 1965.

[2] R. Cox, M. F. Kaashoek, and R. Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, Tech. Rep., 2026.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[4] D. A. Patterson and A. S. Waterman, *The RISC-V Reader: An Open Architecture Atlas*, San Francisco, CA, USA: Strawberry Canyon LLC, 2017.

ANNEX A: COMPLETE SOURCE CODE LISTINGS

A.1 Source code of the kernel counting semaphore implementation (`kernel/sem.c`)

```
#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;
  int value;
};

static struct sem sems[NSEM];

uint64 sys_sem_init(void) {
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

uint64 sys_sem_down(void) {
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);
    release(&s->lock);
    sleep();
    acquire(&s->lock);
    if (killed(myproc())) {
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

uint64 sys_sem_up(void) {
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];
  acquire(&s->lock);
  s->value++;
```

```
  wakeup(s);
  release(&s->lock);
  return 0;
}
```

A.2 Source code of the user semaphore test program (`user/semtest.c`)

```
#include "kernel/types.h"
#include "user/user.h"

int main(void) {
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, semaphore 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf(" pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

A.3 Source code of the concurrent process mixing utility (`user/mix.c`)

```
#include "kernel/types.h"
#include "user/user.h"

void lines(char *s, int uselock) {
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int main(int argc, char *argv[]) {
  int uselock = (argc > 1);
  sem_init(3, 1);
```

```
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

A.4 Complete git patch of xv6 kernel semaphore modifications (`sem.patch`)

```
diff --git a/Makefile b/Makefile
index af1e7a5..d202eeb 100644
--- a/Makefile
+++ b/Makefile
@@ -18,6 +18,7 @@ OBJS = \
   $K/trap.o \
   $K/syscall.o \
   $K/sysproc.o \
+  $K/sem.o \
   $K/bio.o \
   $K/fs.o \
   $K/log.o \
@@ -146,6 +147,8 @@ UPROGS=\
    $U/_grind\
    $U/_wc\
    $U/_zombie\
+   $U/_semtest\
+   $U/_mix\
    $U/_logstress\
    $U/_forphan\
    $U/_dorphan\
diff --git a/kernel/syscall.c b/kernel/syscall.c
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -104,6 +104,9 @@ extern uint64 sys_mkdir(void);
 extern uint64 sys_close(void);
 extern uint64 sys_sync(void);
+extern uint64 sys_sem_init(void);
+extern uint64 sys_sem_down(void);
+extern uint64 sys_sem_up(void);

 static uint64 (*syscalls[])(void) = {
@@ -131,6 +134,9 @@ static uint64 (*syscalls[])(void) = {
   [SYS_close]   = sys_close,
   [SYS_sync]    = sys_sync,
+  [SYS_sem_init] = sys_sem_init,
+  [SYS_sem_down] = sys_sem_down,
+  [SYS_sem_up]   = sys_sem_up,
 };
diff --git a/kernel/syscall.h b/kernel/syscall.h
--- a/kernel/syscall.h
+++ b/kernel/syscall.h
@@ -22,3 +22,6 @@
 #define SYS_close  21
 #define SYS_sync   22
+#define SYS_sem_init 24
+#define SYS_sem_down 25
+#define SYS_sem_up   26
diff --git a/user/user.h b/user/user.h
--- a/user/user.h
+++ b/user/user.h
@@ -27,3 +27,6 @@ int pause(int);
 int uptime(void);
 int sync(void);
```

```
+int sem_init(int, int);
+int sem_down(int);
+int sem_up(int);
diff --git a/user/usys.pl b/user/usys.pl
--- a/user/usys.pl
+++ b/user/usys.pl
@@ -45,3 +45,6 @@ entry("uptime");
 entry("sync");
+entry("sem_init");
+entry("sem_down");
+entry("sem_up");
```